

The Devin Draft

How the app is built — the language, framework, and the AI concepts behind a gamified, AI-assisted upskilling tracker.

A runnable Flask reference implementation · Season 1 · The Founding Five

This playbook explains what powers the app you just ran: the technology choices, why each was made, and the AI concepts the design demonstrates. Everything here maps to code that exists in the repository — nothing aspirational.

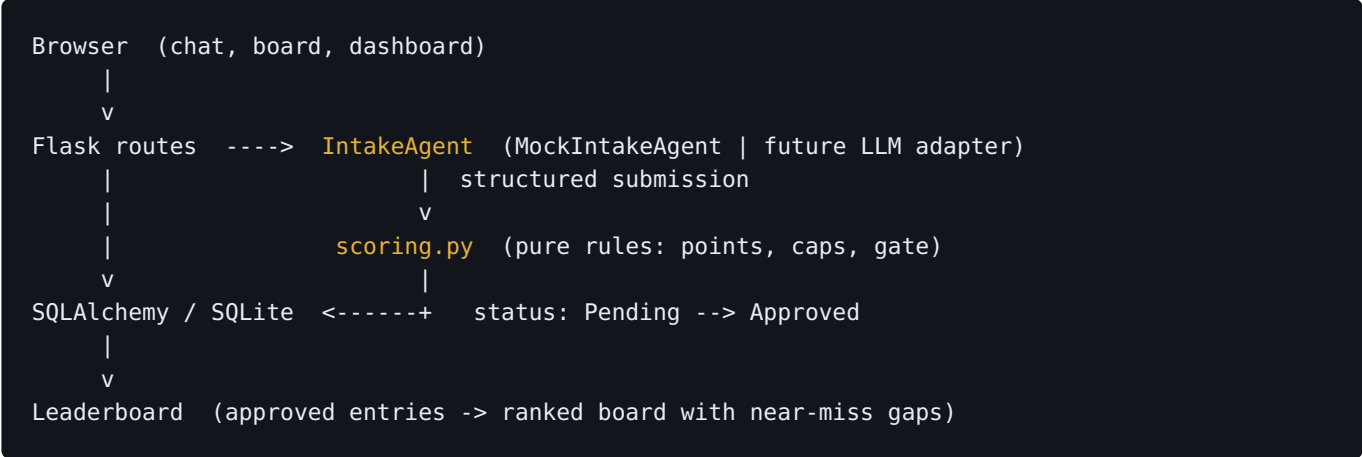
01 Tech stack at a glance

Layer	Technology	Role in the app
Language	Python 3.12	Everything server-side
Web framework	Flask 3	Routing, request handling, page rendering
Database toolkit (ORM)	Flask-SQLAlchemy 3.1	Models and queries as Python objects
Database	SQLite	A single file on disk — zero setup
Templating	Jinja2	Server-rendered HTML (ships with Flask)
Frontend	HTML, CSS, vanilla JS	The board, dashboard, and chat — no build step
Testing	pytest	21 automated tests

Why this stack

The whole app runs with three installed packages and no external services — no API keys, no cloud database, no Node build. That is a deliberate choice: a reviewer can clone it and have it running in under a minute. SQLite means the "database" is one file; Flask keeps the surface area small; vanilla JS avoids a toolchain. It is the smallest stack that still demonstrates real engineering.

02 How the pieces fit together



Four concerns are kept deliberately separate so each can be reasoned about and tested alone. Dependencies point *inward* toward the pure rules — the scoring logic has no idea Flask or SQLite exist, and the agent has no idea how points are stored.

Module	Responsibility
app/scoring.py	The rules: points per activity, the fluency cap, the qualification gate, leaderboard ranking. Pure functions, no framework.
app/agent.py	The intake agent: an interface plus a deterministic mock that interviews the user.
app/models.py	Database tables (Submission, Season) as Python classes.
app/routes.py	Pages and the JSON API that ties it all together.

03 Core engineering concepts

Application factory & separation of concerns

The app is built by a `create_app()` function rather than a global object, so tests spin up an isolated instance with an in-memory database. Each module owns one job, which is what makes the 21 tests possible.

Pure functions for the rules

All scoring lives in framework-free functions. Given the same input they always return the same output, with no database or network involved — the reason the rules can be exhaustively unit-tested.

Server-computed trust boundary

The browser never sends a point value. It sends an activity type and two flags; the server computes the score. Even the *track* is derived from the activity, so a submission cannot claim points under the wrong category. Never trust the client with values it should not control.

Human-in-the-loop gate

Every entry is created as Pending. Only after a human sets it to Approved does it reach the leaderboard. The AI proposes; a person disposes.

04 The AI concepts this app demonstrates

The app is intentionally an **AI-assisted** system, not just a form with a database. These are the concepts it puts into practice — the same ideas the upskilling program itself is trying to teach.

CONCEPT · CONVERSATIONAL INTAKE AGENT

An agent that interviews, instead of a form that waits

Rather than a static form, an **agent** asks one question at a time, adapts the next question to the answer (a POC triggers different questions than a course), and produces a clean structured record at the end. This is the difference between *data entry* and an *agentic capture flow*.

CONCEPT · THE AGENT BEHIND AN INTERFACE

Swappable intelligence: mock today, LLM tomorrow

The agent is defined as an abstract interface (`IntakeAgent`) with two methods, `start()` and `step()`. The shipped `MockIntakeAgent` is deterministic and needs no API key, so the app always runs. A real large-language-model agent implements the same two methods and drops in without changing routes, storage, or tests. This is the **adapter pattern** applied to AI: the model is a detail, not the architecture.

CONCEPT · INPUT VALIDATION / GROUNDING

Refusing vague input

When logging a POC, the agent checks that the stated business value contains a real metric. "Improves productivity" is rejected; "cut build time from 40 to 12 minutes" is accepted. In AI terms this is a lightweight **grounding / validation** step — forcing claims to be specific and measurable before they enter the system. In the mock this is a rule; with an LLM it becomes a prompt instruction, but the principle is identical.

CONCEPT · HUMAN-IN-THE-LOOP (HITL)

AI proposes, a human decides

No seat is ever awarded by the software. The agent structures and scores a submission; a lead approves it. **Human-in-the-loop** is the discipline of keeping a person at the decision point for anything consequential — the exact principle an AI adoption program should model in its own tooling.

CONCEPT · DETERMINISTIC VS. GENERATIVE

Knowing when *not* to use a model

Scoring is deterministic arithmetic, so it lives in tested pure functions, not in a model. The judgment-and-conversation part is where intelligence adds value. Good AI engineering is partly knowing which problems need a model and which are better as plain, verifiable code.

Carrying context across turns

A conversation needs memory of what was already asked. The agent keeps a small **state** object (carried in the web session) recording answers so far and the current question. Managing this per-turn state is the foundation every multi-step agent is built on, whether rule-based or model-driven.

05 Behavioural-design concepts

The "game" is engineering too — three well-known levers, each realised in the scoring code.

Lever	How the app expresses it
Scarcity	A fixed, visible seat count on the dashboard.
Loss aversion	Seats framed as renewable leases, not gifts (concept layer).
Near-miss feedback	The board shows each person's exact distance to the cut and who is just ahead.

06 Running it & the test suite

The three things it does

- **Capture** — the chat agent turns a conversation into a structured, validated submission.
- **Score & gate** — pure functions compute points, apply the 25% fluency cap, and decide qualification (250 points + all three tracks).
- **Rank** — approved entries become a live board with ranks and near-miss gaps.

Test coverage. 21 pytest tests cover the scoring rules (base points, bonuses, the fluency cap, the qualification gate, ranking and tie-breaks) and the API/agent flow (vague-value pushback, pending-stays-off-board, approval-promotes-to-board). Tests are the evidence the rules do what the playbook claims.

07 Glossary

Term	Plain meaning
Framework (Flask)	Pre-written plumbing for handling web requests so you don't build it from scratch.
ORM (SQLAlchemy)	Lets you work with database rows as Python objects instead of writing SQL.
Agent	Software that carries out a multi-step task through a dialogue or a sequence of decisions.

Interface / adapter

An agreed set of methods, so one component can be swapped for another that fits the same shape.

Human-in-the-loop	Keeping a person at the point where a consequential decision is made.
Grounding	Making the system insist on specific, verifiable input rather than vague claims.
Deterministic	Same input always gives the same output — predictable and testable.
Pure function	A function whose output depends only on its inputs, with no side effects.